

Project 2: Bayesian Logistic Regression for Binary Classification with Numerical Integration & Monte Carlo

Due: 22 May 23:59

You are required to submit a report and all the codes for the project.

Given a dataset $\mathcal{D} = \{(\vec{x}_i, y_i)\}_{i=1}^N$, where $\vec{x}_i \in \mathbb{R}^d$ is a feature vector (may include a bias term, so the first component is usually 1) and $y_i \in \{+1, -1\}$ is the class label, logistic regression models the probabilities that $y_i = \pm 1$ as:

$$P(y_i = +1 \mid \vec{x}_i, \vec{w}) = \sigma(\vec{w}^T \vec{x}_i) = \frac{1}{1 + e^{-\vec{w}^T \vec{x}_i}},$$

$$P(y_i = -1 \mid \vec{x}_i, \vec{w}) = 1 - \sigma(\vec{w}^T \vec{x}_i) = 1 - \frac{1}{1 + e^{-\vec{w}^T \vec{x}_i}} = \frac{1}{1 + e^{\vec{w}^T \vec{x}_i}}.$$

The two cases can be combined into a single expression:

$$P(y_i \mid \vec{x}_i, \vec{w}) = \sigma(y_i \vec{w}^T \vec{x}_i).$$

Assuming the data points are independent, the likelihood is:

$$p(\mathcal{D} \mid \vec{w}) = \prod_{i=1}^N \sigma(y_i \vec{w}^T \vec{x}_i).$$

We treat the weight vector $\vec{w}$ as a random variable. A **prior distribution** $p(\vec{w})$ encodes our beliefs before seeing the data. A common choice is a zero-mean Gaussian prior:

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, \alpha^{-1} I) = \left(\frac{\alpha}{2\pi}\right)^{d/2} \exp\left(-\frac{\alpha}{2} \vec{w}^T \vec{w}\right),$$

where $\alpha > 0$ is a precision parameter (inverse variance).

Remark: The general multivariate normal distribution is

$$\mathcal{N}(\vec{w} \mid \vec{\mu}, \Sigma) = \left(\frac{1}{2\pi}\right)^{\frac{d}{2}} \det(\Sigma)^{-1/2} \exp\left(-\frac{1}{2} (\vec{w} - \vec{\mu})^T \Sigma^{-1} (\vec{w} - \vec{\mu})\right).$$

Part 1: 1 D Rejection to Sample Normal Distributions

The Laplace (double exponential) distribution is given by

$$p(w) = \frac{\lambda}{2} e^{-\lambda|w|}, \quad -\infty < w < \infty, \lambda > 0.$$

To sample the standard normal distribution

$$\mathcal{N}(w|0, 1) = \frac{1}{\sqrt{2\pi}} e^{-w^2/2}, \quad -\infty < w < \infty$$

by rejection method, let's use the unnormalized Laplace distribution

$$f(w) = A e^{-\lambda|w|}, \quad -\infty < w < \infty, \lambda > 0$$

as the comparison function.

- (a) Show that for $f(w)$ to be a valid comparison function,

$$A \geq \frac{1}{\sqrt{2\pi}} e^{\lambda^2/2}.$$

- (b) Evaluate the theoretical acceptance rate (the fraction of points being accepted). Express your answer in terms of A and/or λ .

- (c) Show that the optimal choice is

$$f(w) = \sqrt{\frac{e}{2\pi}} e^{-|w|}.$$

What is the acceptance rate for this choice?

- (d) Write a code to sample $\mathcal{N}(0, 1)$ by rejection method using the above comparison function. The code should be able to generate any required number of points $\sim \mathcal{N}(0, 1)$.
- (e) Run your code for trial and plot the distribution to verify that it looks like a standard normal distribution. Also check whether your code gives the acceptance rate predicted in (c).
- (f) State how you can use the code to sample the general normal distribution $\mathcal{N}(\mu, \sigma^2)$. Modify your code so that it can sample any required number of points $\sim \mathcal{N}(\mu, \sigma^2)$ for any given μ and σ .

Part 2: Numerical Integration for Marginal Likelihood

For a **prior distribution** $p(\vec{w})$, **Bayes' theorem** gives the **posterior distribution**:

$$p(\vec{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \vec{w}) p(\vec{w})}{p(\mathcal{D})},$$

where the denominator is the **marginal likelihood** (also called the **evidence**):

$$p(\mathcal{D}) = \int_{\mathbb{R}^d} p(\mathcal{D} | \vec{w}) p(\vec{w}) d\vec{w},$$

which can also be interpreted as the reciprocal of the normalization constant of the distribution $\propto p(\mathcal{D} \mid \vec{w}) p(\vec{w})$.

The marginal likelihood is in general a multi-dimensional integral. For low dimensions it can be computed with numerical quadrature; for higher dimensions we usually turn to Monte Carlo methods.

We consider a simple model with only two parameters: bias w_0 and one feature weight w_1 . Take the prior distribution to be

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, I) = \frac{1}{2\pi} \exp\left(-\frac{w_0^2 + w_1^2}{2}\right).$$

(a) Generating Synthetic Classification Data:

We choose true weight $\vec{w}_{\text{true}} = (w_0 \ w_1)^T = (-1 \ 1)^T$.

Data: Sample $N = 500$ $x_i \sim \mathcal{N}(-\frac{w_0}{w_1} = 1, 1)$, add bias 1, i.e., $\vec{x}_i = (1 \ x_i)^T$.

Label $y_i = \text{sign}(\vec{w}_{\text{true}}^T \vec{x}_i + \epsilon_i)$, where $\epsilon_i \sim \mathcal{N}(0, 4)$ is to simulate noise.

The marginal likelihood when we integrate out w_1 with fixed $w_0 = -1$ is:

$$p(\mathcal{D} \mid w_0) = \int_{-\infty}^{\infty} \left[\prod_{i=1}^N \sigma(y_i(w_0 + w_1 x_{i,1})) \right] \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{w_1^2}{2}\right) dw_1.$$

- (b) Evaluate the above integral by trapezoidal rule. Set the tolerance (relative error) to be 10^{-6} . Use the error estimation formula in the lecture notes. What is the total number of integrand evaluations?
- (c) Evaluate the above integral by Romberg integration. Set the tolerance (relative error) to be 10^{-6} . Use the error estimation formula in the lecture notes. What is the total number of integrand evaluations? How does it compare with that in (b)?

If w_0 is also integrated out, the marginal likelihood is

$$p(\mathcal{D}) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \left[\prod_i \sigma(y_i(w_0 + w_1 x_{i,1})) \right] \frac{1}{2\pi} \exp\left(-\frac{w_0^2 + w_1^2}{2}\right) dw_0 dw_1.$$

- (d) Evaluate the above double integral by Simpson's rule. Set the tolerance (relative error) to be 10^{-6} . Use the error estimation formula in the lecture notes.

- (e) Evaluate the above double integral with error estimation by crude Monte Carlo (MC) integration. Here “crude” means using points with prior distribution

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, I) = \frac{1}{2\pi} \exp\left(-\frac{w_0^2 + w_1^2}{2}\right).$$

Remark: In (b), (c), and (d), you may handle the improper integrals over infinite interval by transforming it to a finite interval. Alternatively, because the Gaussian prior decays quickly, you may also truncate the integration domain to $[-L, L]$ with L chosen so that the contribution of the tails is much smaller than the tolerance) (e.g., $L = 6$). Check that increasing L does not change the integral within the tolerance.

Part 3: Metropolis Algorithm, Simulated Annealing, and Importance Sampling (Variance Reduction)

To evaluate integrals involving the posterior distribution

$$\int f(\vec{w}) p(\vec{w} \mid \mathcal{D}) d\vec{w},$$

one may (without knowing the marginal likelihood) draw samples by the Metropolis algorithm, which constructs a Markov chain whose stationary distribution is the target $p(\vec{w} \mid \mathcal{D})$.

- (a) Sample 10000 points by the Metropolis algorithm. Use a symmetric Gaussian proposal:

$$\vec{w}' = \vec{w} + \vec{\delta}, \quad \vec{\delta} \sim \mathcal{N}(\vec{0}, \sigma_p^2 I).$$

Tune σ_p so that the acceptance rate is $\sim 30\%$.

Plot the distribution you've sampled and compare it with another plot of the function $p(\vec{w} \mid \mathcal{D})$ using the value of $p(\mathcal{D})$ you obtained in Part 2.

More generally, one may write the integral as

$$\int f(\vec{w}) p(\vec{w} \mid \mathcal{D}) d\vec{w} = \int f(\vec{w}) \frac{p(\vec{w} \mid \mathcal{D})}{q(\vec{w})} q(\vec{w}) d\vec{w}.$$

If we can find a $q(\vec{w})$ which is easy to sample and approximately proportional to $f(\vec{w}) p(\vec{w} \mid \mathcal{D})$, the variance can be dramatically reduced.

Let's consider the case when the variance of $f(\vec{w})$ is negligible. Then one may let $q(\vec{w})$ be a Gaussian with an appropriate width and peaked at the global maximum of $p(\vec{w} | \mathcal{D})$, also known as the **maximum a posteriori (MAP)**

$$\vec{w}_{\text{MAP}} = \arg \max_{\vec{w}} p(\vec{w} | \mathcal{D}).$$

To find MAP we can turn the problem into an optimization by using simulated annealing with the energy function defined to be

$$E(\vec{w}) = -\log p(\vec{w} | \mathcal{D}).$$

- (b) Find the MAP by simulated annealing.
- (c) Calculate the marginal likelihood

$$p(\mathcal{D}) = \int p(\mathcal{D} | \vec{w}) p(\vec{w}) d\vec{w} = \int \frac{p(\mathcal{D} | \vec{w}) p(\vec{w})}{q(\vec{w})} q(\vec{w}) d\vec{w}$$

by MC integration using points with distribution $q(\vec{w})$, which is a Gaussian peaked at the MAP in (b). The width can be roughly estimated using the sample variance in (a). State both the value of the integral and the error estimation and compare them with that in Part 2 (d) and (e).

Remarks on Coding

The products

$$\prod_{i=1}^N \sigma(y_i \vec{w}^T \vec{x}_i)$$

may lead to underflow or numerical instability. Therefore, it's better to work with logarithms and exponentiate only when needed.

You may use the helper functions below if needed. They are written in vectorized form for arbitrary dimension d , so that you can utilize them if you want to apply the techniques you learned in this project to handle real data in higher dimensions.

The function `log1p(x)` is defined by

$$\text{log1p}(x) = \log(1 + x).$$

It is designed to be accurate for very small $|x|$, where directly computing $\log(1 + x)$ would suffer from catastrophic cancellation (the $1 + x$ operation loses precision). The `log1p` function uses a dedicated algorithm to preserve the full precision of x when $x \approx 0$.

Helper Functions:

```
def log_likelihood(w, X, y):  
    """  
    Compute log-likelihood for logistic regression.  
    w: (d,) coefficient vector  
    X: (N, d) design matrix  
    y: (N,) labels in {-1, +1}  
    Returns scalar log-likelihood.  
    """  
    z = y * (X @ w)  
    # Use log1p for numerical stability: log(sigma(z)) = -log(1+exp(-z))  
    # For large positive z, exp(-z) underflows safely to 0, log1p(0)=0 -> correct.  
    # For large negative z, exp(-z) huge -> but then we use the alternative formula:  
    # sigma(z) = 1/(1+exp(-z)) = exp(z)/(1+exp(z)) => log(sigma(z)) = z - log(1+exp(z))  
    # We use the stable implementation: -log(1+exp(-z))  
    # However, for very negative z, -log(1+exp(-z)) ≈ -log(exp(-z)) = z
```

```

# So we can compute: log_sigma = -np.log1p(np.exp(-z)) works fine.
log_sigma = -np.log1p(np.exp(-z))
return np.sum(log_sigma)

def log_prior(w, alpha=1.0):
    """
    Zero-mean Gaussian prior with precision alpha (alpha=1 corresponds to variance 1).
    Returns log of  $N(w|\theta, \alpha^{-1} I) = -(\alpha/2) w^T w - (d/2) \log(2\pi/\alpha)$ 
    We can drop constants as they cancel in ratios, but for evidence we need them.
    For Metropolis and SA we can drop constants.
    """
    d = len(w)
    return -0.5 * alpha * np.dot(w, w) - 0.5 * d * np.log(2*np.pi/alpha)

def log_posterior(w, X, y, alpha=1.0):
    """Unnormalized log posterior: log likelihood + log prior."""
    return log_likelihood(w, X, y) + log_prior(w, alpha)

# For convenience, we define a function that returns the unnormalized posterior density
def unnormalized_posterior(w, X, y, alpha=1.0):
    return np.exp(log_posterior(w, X, y, alpha))

```